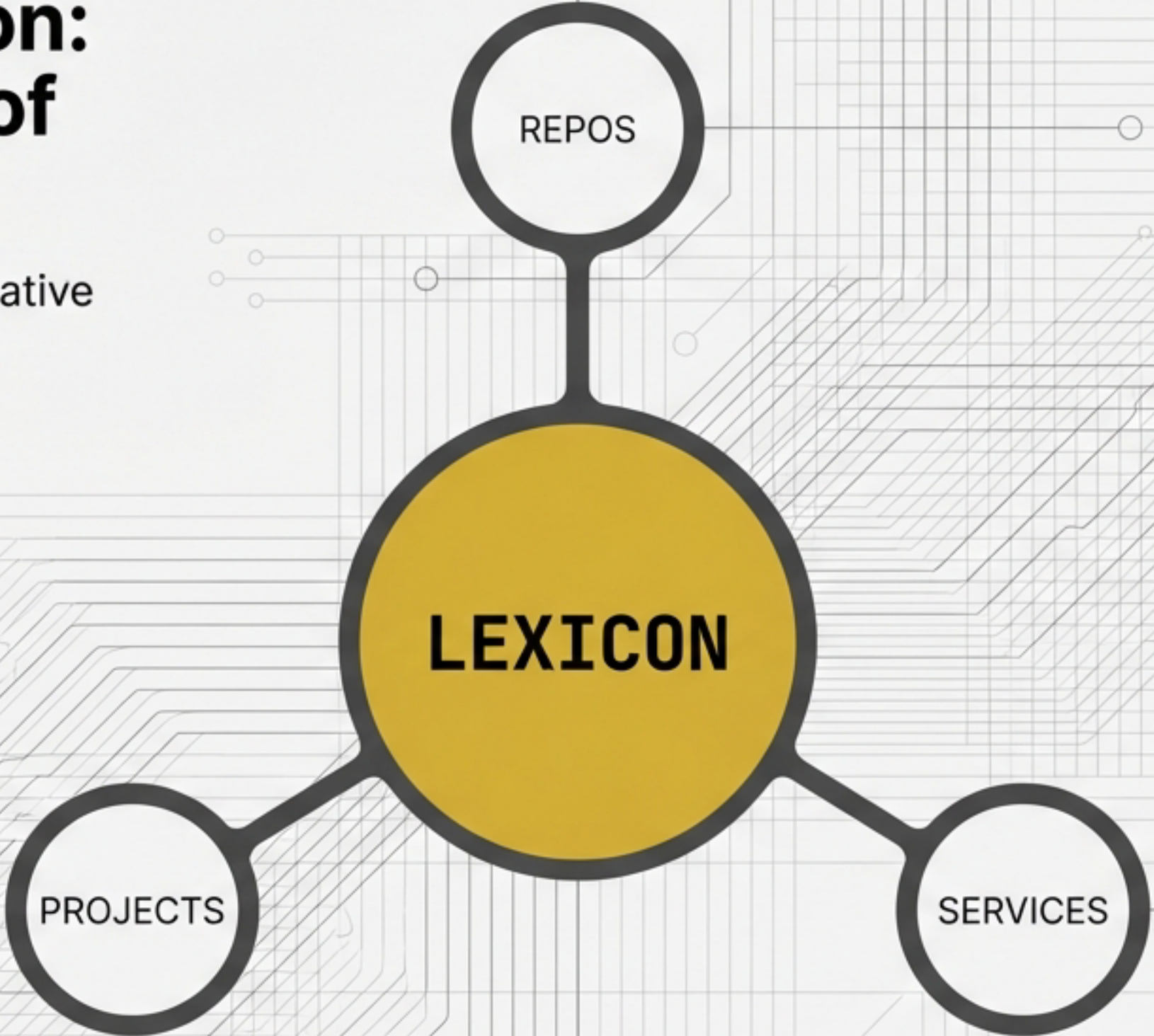


Issues-FS Lexicon: The Root Graph of the Ecosystem

Architecture v2.0 | From Authoritative
Schemas to Connected Anchors



STATUS:	Draft
DATE:	2026-02-05
DEPENDENCIES:	issues-fs__thinking-in-graphs v1.0, osbot-utils

THE TWO-DEPENDENCY INVARIANT

STRUCTURE (How We Build)



osbot-utils

- Type_Safe patterns
- ✕ Structural code
- + Utility libraries

MEANING (What We Mean)



Issues-FS__Lexicon

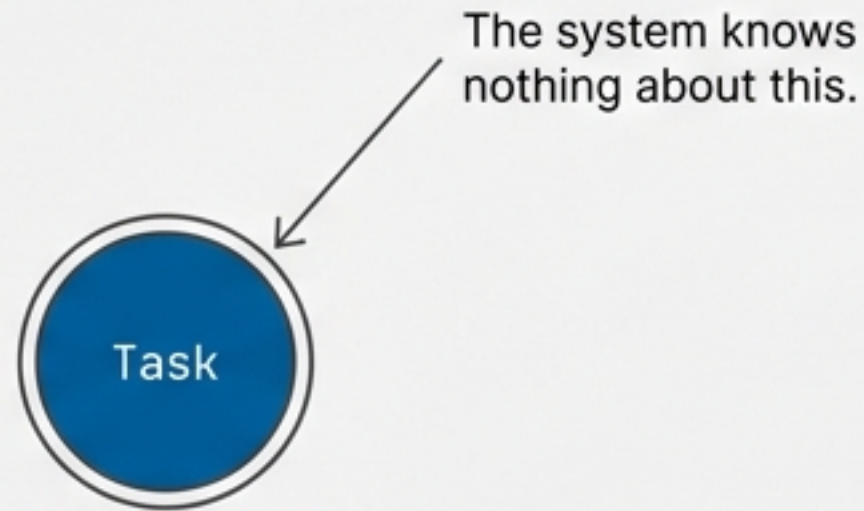
- Shared vocabulary
- ✕ Reference graphs
- + Anchor nodes

Universal Availability: Any repo can link to shared anchor nodes without importing heavyweight dependencies. Connectivity starts from day one.

MEANING REQUIRES CONNECTIVITY

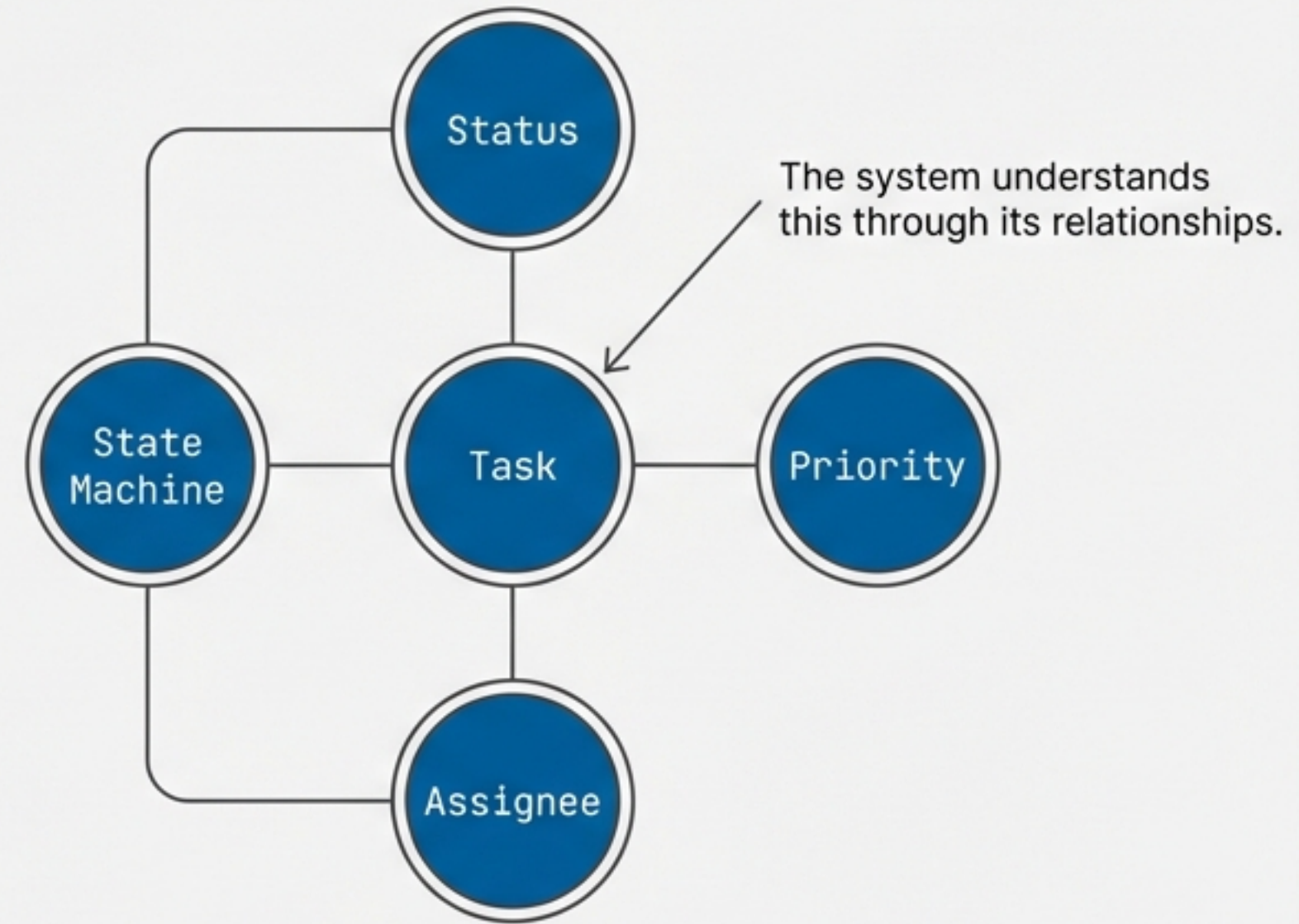
A

Isolation (Zero Meaning)



B

Connectivity (High Meaning)



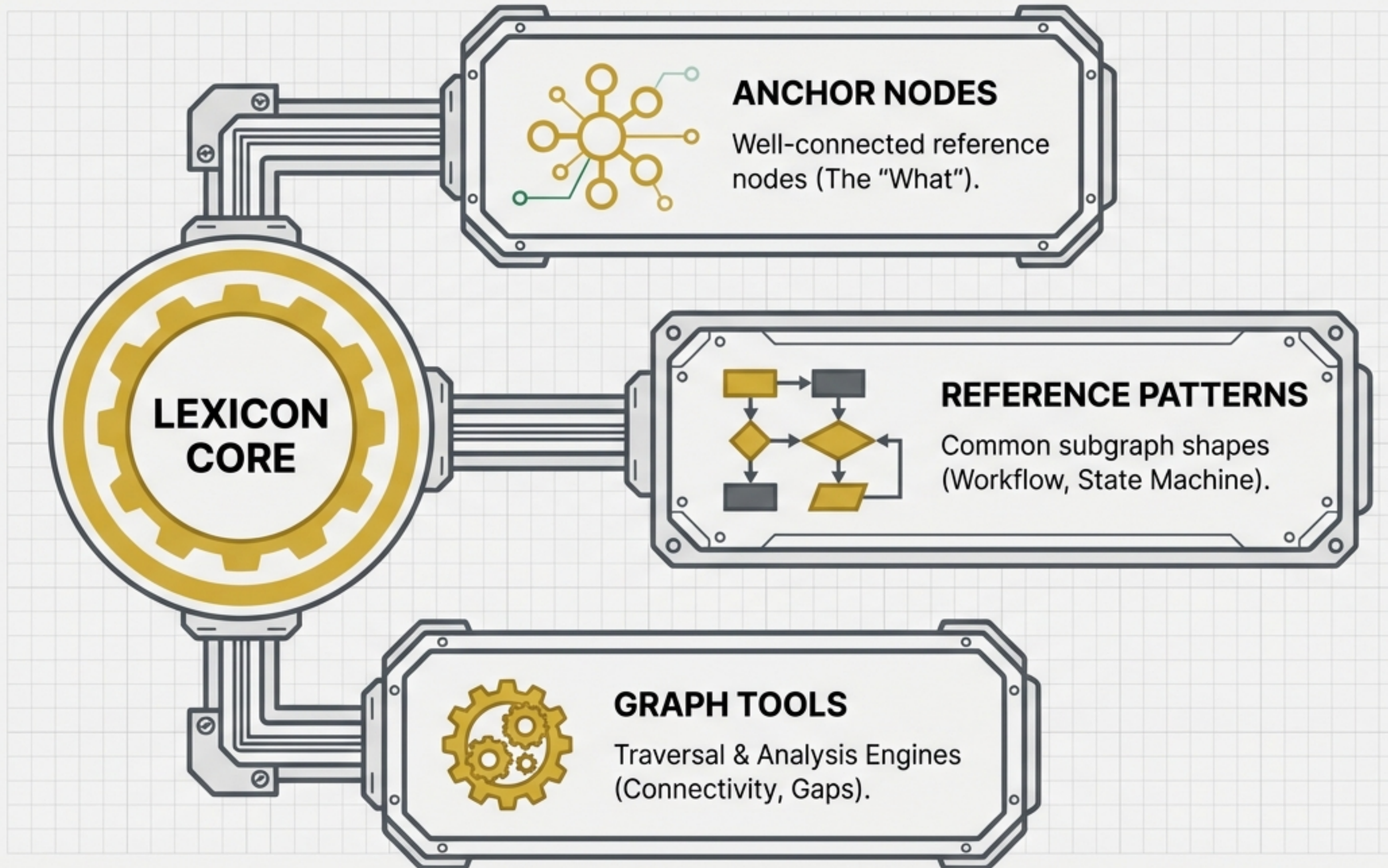
The Core Principle: A node has no inherent meaning. Meaning is defined strictly by the edges you can trace from it.

THE ANCHOR PROBLEM: WHY WE CANNOT RELY ON LOCAL DEFINITIONS



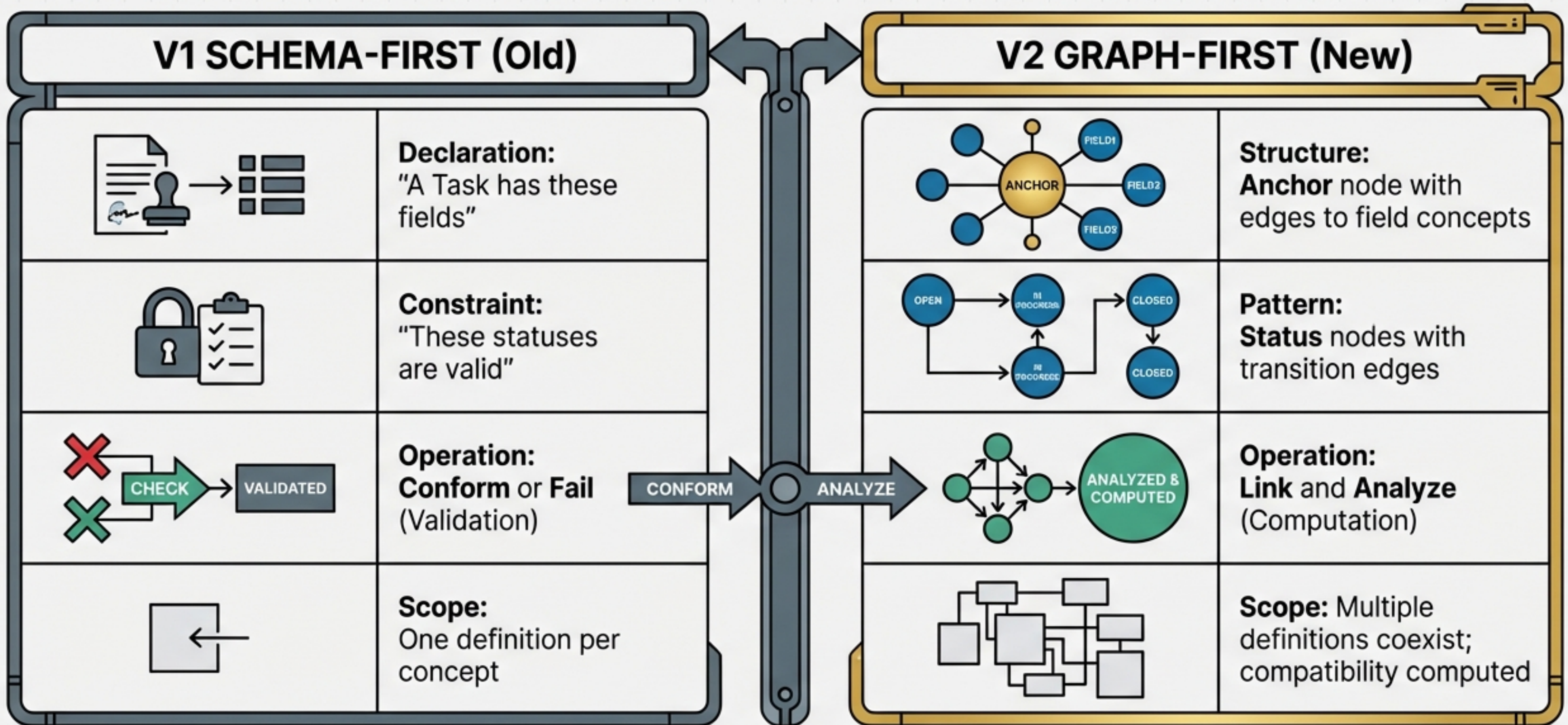
The Scenario: Project-6 and Project-7 both define a 'Task'. They are locally meaningful but globally opaque.
The Failure State: Without a shared reference anchor, analysis tools see these as **"Islands of Logic"**.

THE LEXICON IS A ROOT GRAPH, NOT A SCHEMA REGISTRY



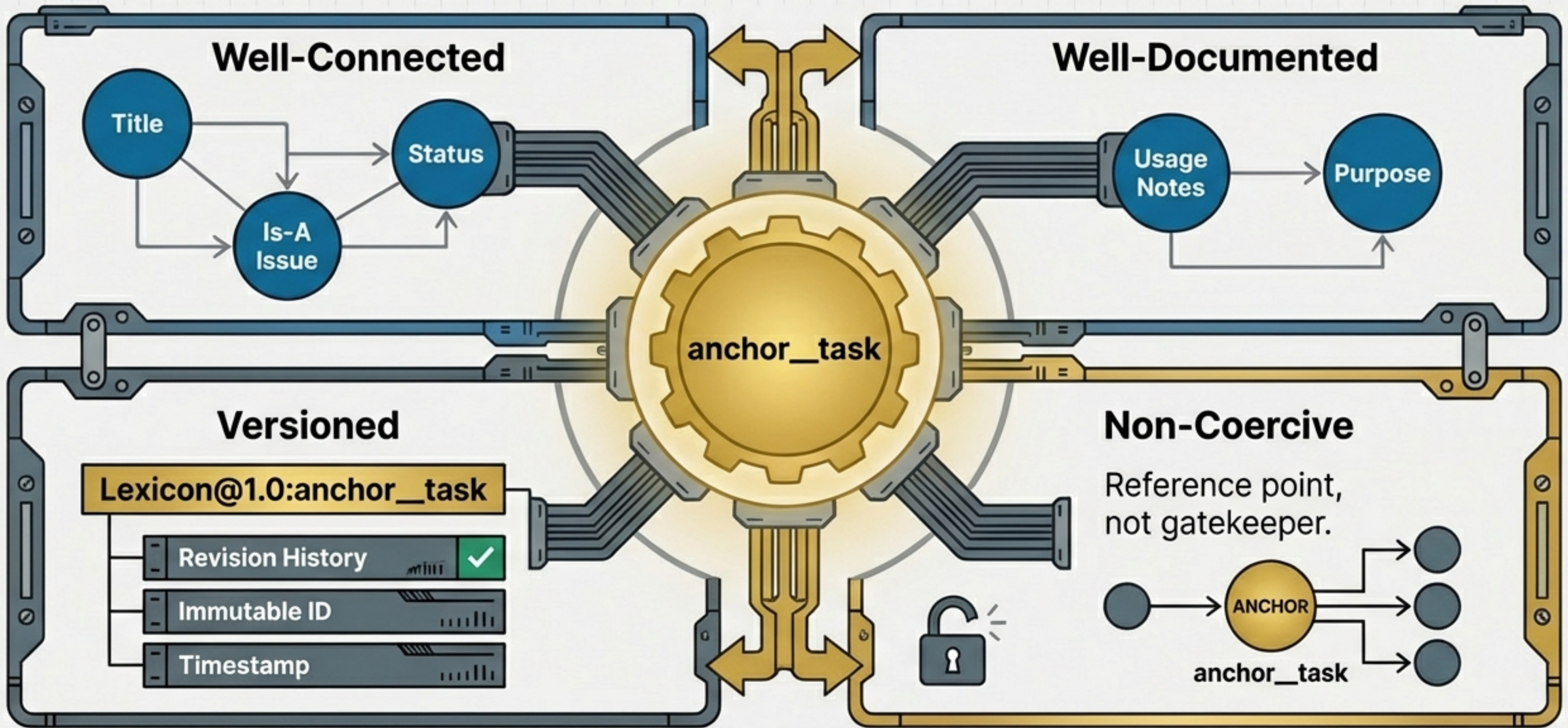
The Lexicon is **not a gatekeeper**. It is the most well-connected graph in the ecosystem. Others link to it to increase their own confidence and interoperability.

THE PARADIGM SHIFT: FROM VALIDATION TO ANALYSIS



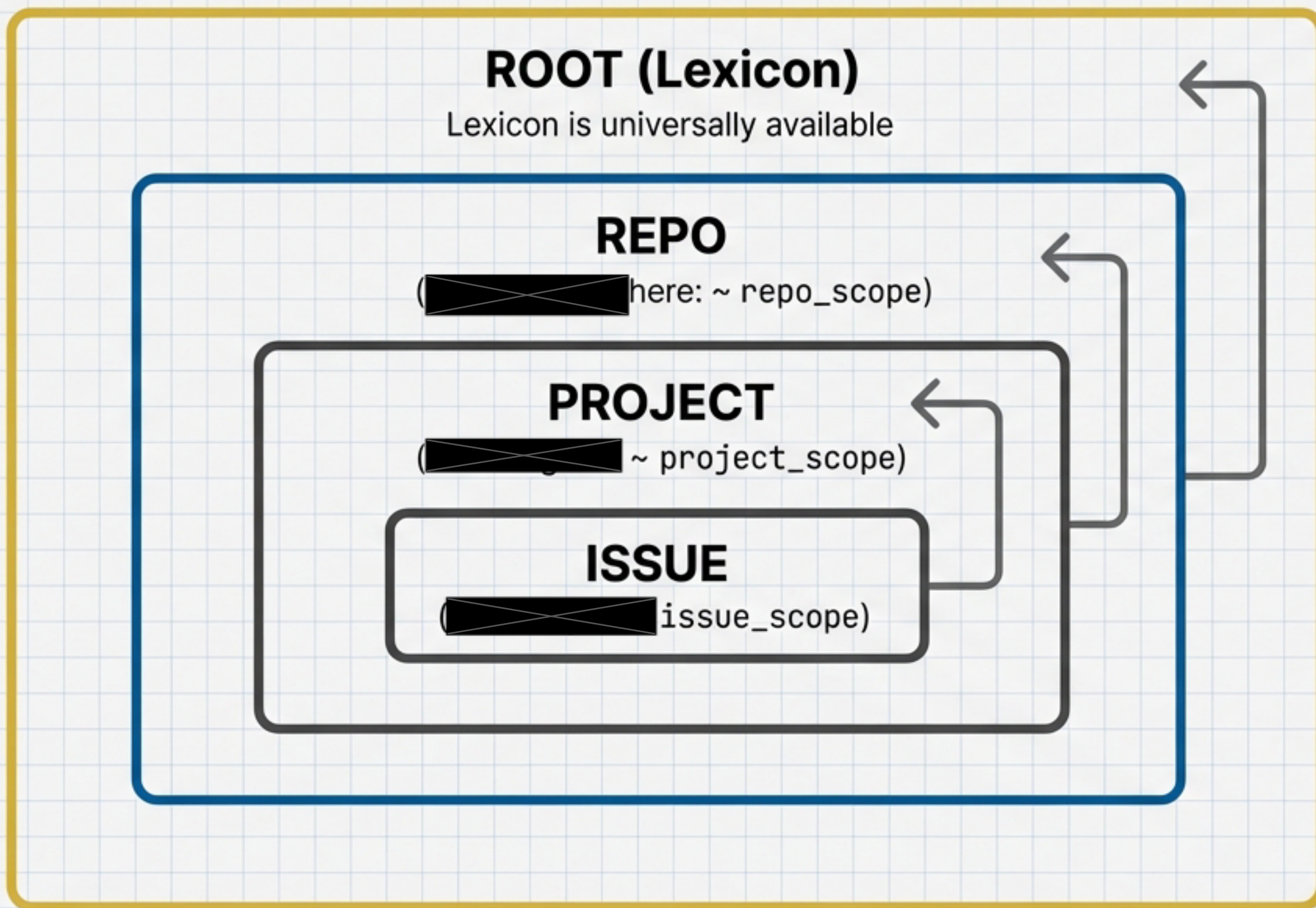
We replace `test\_\_consistency` with `test\_\_graph\_integrity`.

ANATOMY OF AN ANCHOR NODE



The anchor is not a box containing text. It is the center point of a rich subgraph.

THE FRACTAL MODEL: EVERY SCOPE IS A POTENTIAL LEXICON



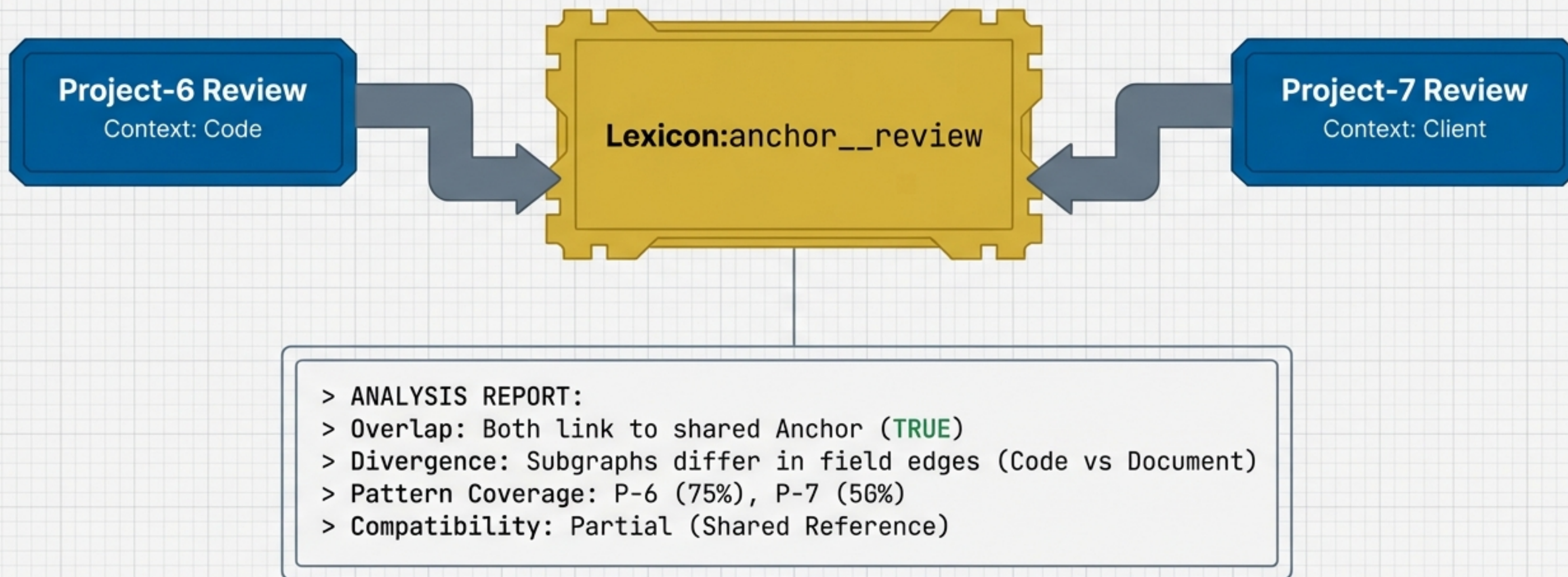
SCOPE RESOLUTION LOGIC

1. Local match? Use it.
(e.g., anchor__task in ISSUE)
2. Parent match? Use it.
(e.g., anchor__task in PROJECT)
3. Root match? Use it.
(e.g., anchor__task in ROOT)
4. No match? Report "No definition found" (Do not fail).

KEY INSIGHT:

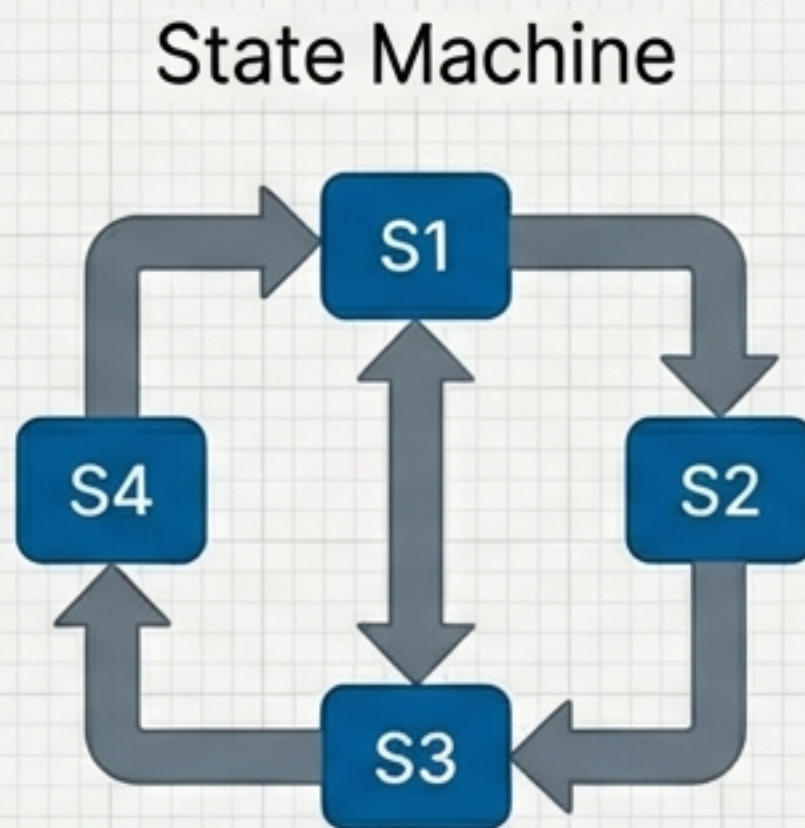
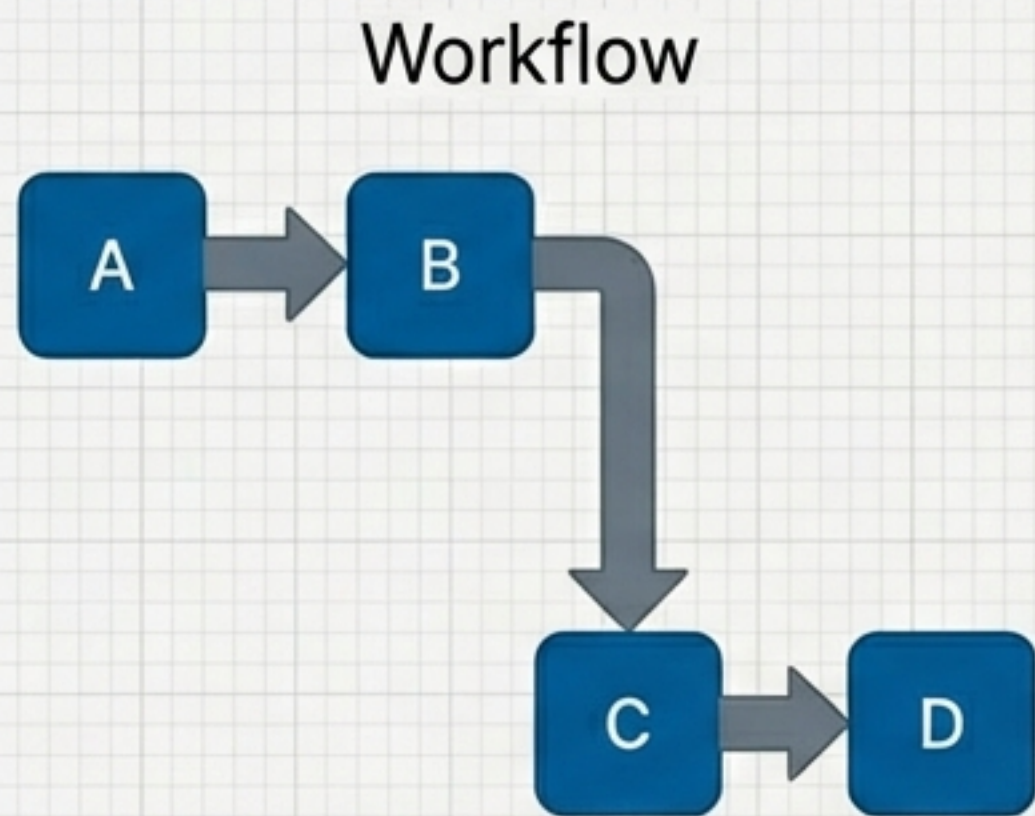
The Lexicon is the Root scope. It is universally available, but not 'correct' by authority. A project definition is just as valid.

Handling Divergence: Computation over Conflict



 Insight Text: The system surfaces this nuance; it does not resolve it. Resolution is a human decision.

Reference Patterns: Recognising Shapes in the Graph



What they are: Recognisable subgraph shapes, not templates.

How they work: Analysis tools examine a subgraph and report: "Matches Workflow pattern at 4/5 steps".

Flexibility: You build your graph; the tools "recognise" the pattern within it.

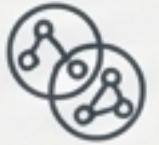
The Analysis Toolkit: Making the Graph Operational

`analysis__connectivity`



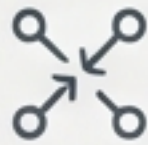
Computes density of connections.
“Linked to **4/7 reference edges**”.

`analysis__compatibility`



Computes **subgraph overlap**
between two nodes.

`analysis__gaps`



Identifies **missing edges** to
increase confidence.

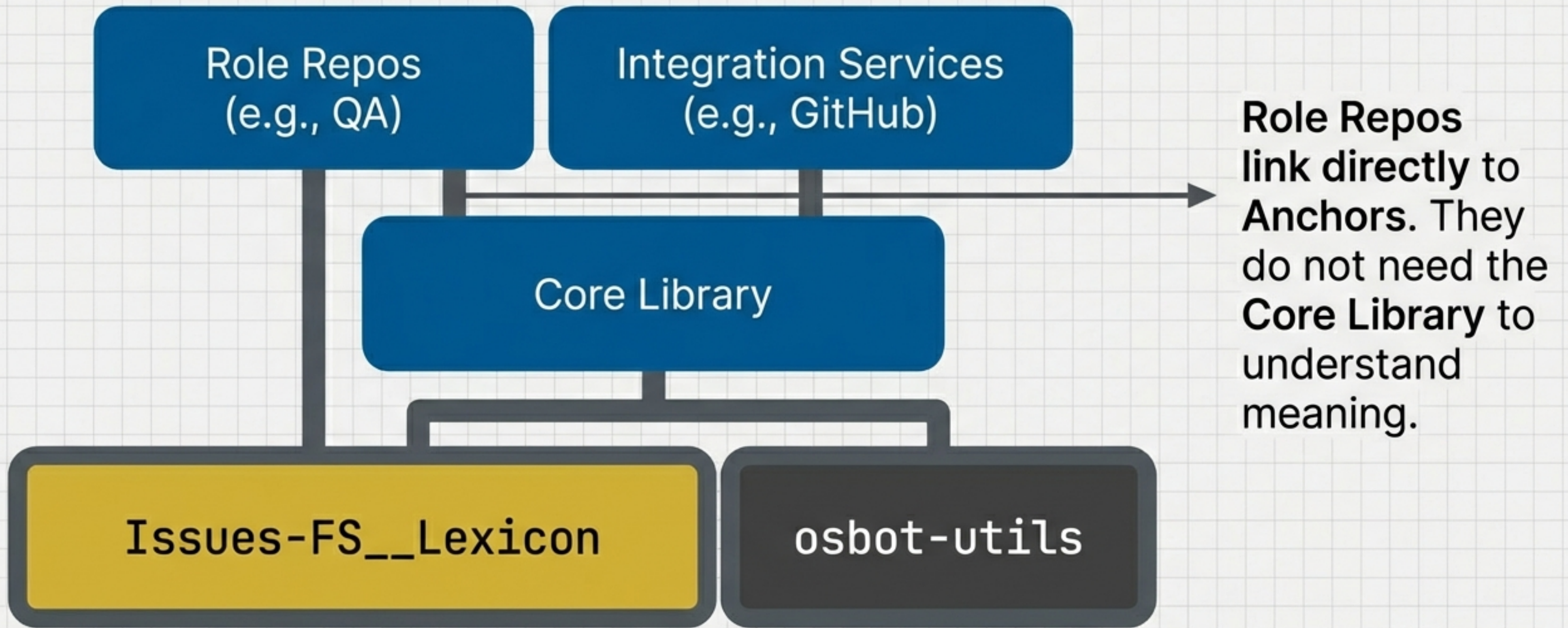
`analysis__scope_resolver`



Walks the **scope chain** to locate
definitions.

These tools ship with the Lexicon and work on ANY graph (Project, Repo, or Root).

Ecosystem Integration

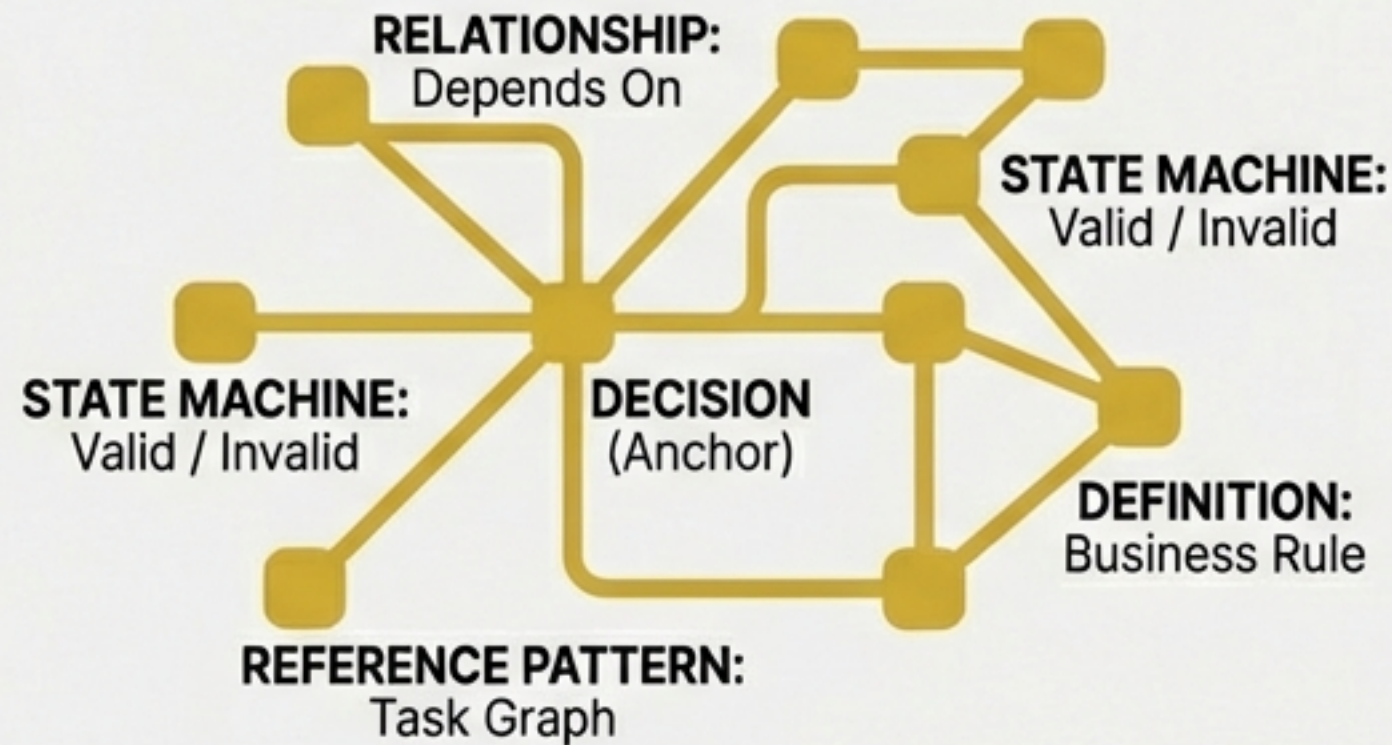


Updated Dependency Graph: The Lexicon is a dependency for every repository.

Distinguishing Concept from Implementation

Lexicon vs. Client_Python

The Lexicon (Concept)



Owns what a Decision IS.
Relationships, State Machines, Reference Patterns.

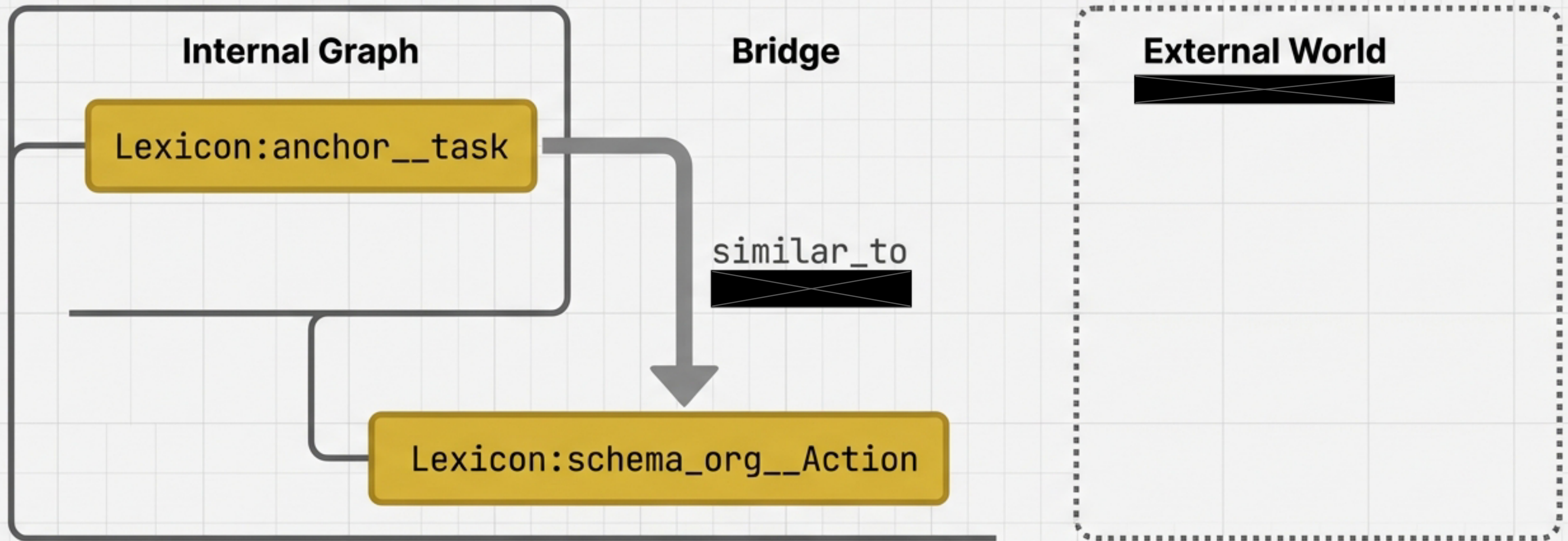
Client_Python (Implementation)

```
class DecisionPayload(BaseModel):  
    decision_id: str  
    payload_data: Dict[str, Any]  
    execution_status: str  
    # ... implementation details ...  
  
    {  
        "decision_id": "dec_12345",  
        "payload_data": {  
            "input_a": 10,  
            "input_b": 20  
        },  
        "execution_status": "PENDING"  
    }
```

Owns what fields a Decision HAS (API Payload).
Request shape, Execution logic.

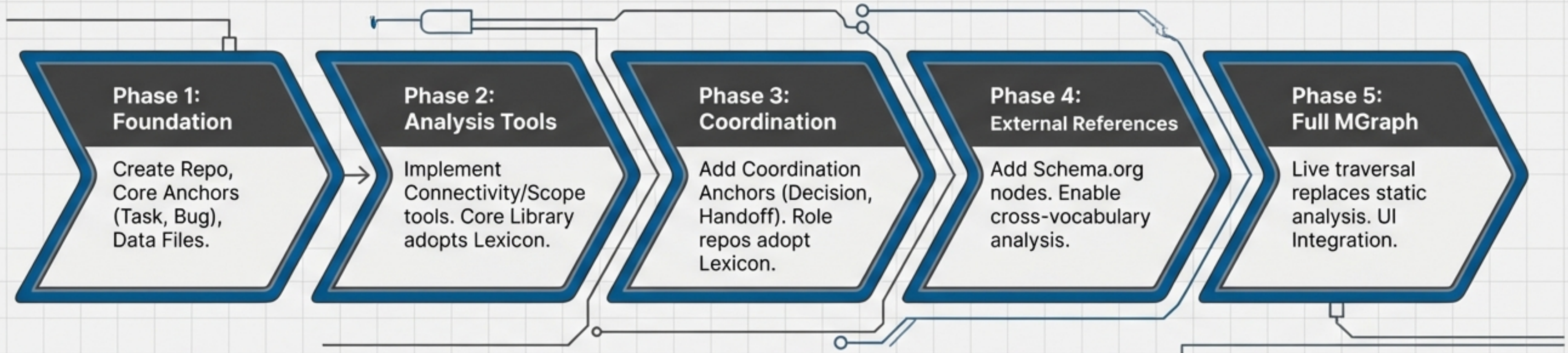
Client_Python depends on Lexicon. API schemas reference Anchor Nodes to track divergence.

External References as First-Class Nodes



We do **not** import Schema.org at runtime. We maintain curated nodes **within** the Lexicon graph.
Why? Allows versioning, enrichment, and divergence tracking.
Path: Local Node -> Lexicon Anchor -> Schema.org Node.

Migration Strategy: From Foundation to MGraph



Honest Uncertainty: The tools report what the graph supports and what it doesn't. We do not fill gaps with assumptions.